Experimental Setup

Experimental Setup I

This section details the complete experimental configuration used to generate the results in this study. Every parameter value below is injected programmatically from `config.yaml` and the analysis pipeline — no value is hardcoded in the manuscript source.

Problem Definition I

The optimization target is the quadratic objective function defined in eq. 1:

$$f(x) = \frac{1}{2} x^T A x - b^T x \tag{1}$$

with $A = [(1.0,)]$ and $b = [1.0]$, yielding the analytical optimum at $x^* = 1.0$ with $f(x^*) = -0.5$.

Parameter Space I

The experiment systematically varies the gradient descent step size across 6 values:

Parameter Space I

- $\alpha = 0.01$ (conservative)
- $\alpha = 0.1$ (conservative)
- $\alpha = 0.5$ (near-optimal)
- $\alpha = 1.0$ (near-optimal)
- $\alpha = 1.5$ (aggressive)
- $\alpha = 2.5$ (divergent (expected unstable for H = I))

Parameter Space I

All runs start from the initial point $x_0 = 0.0$ and use a convergence tolerance of $\|\nabla f\| < 10^{-8}$ with a maximum iteration limit of $N_{\max} = 1000$.

Numerical Stability Grid I

To validate robustness, the optimizer is exercised across a grid of 8 starting points and 6 step sizes, producing 48 total evaluations. This finite sweep measures sensitivity to the selected initial points and step sizes; it does not establish convergence outside the contractive region or before every iteration cap. The stability metric is computed for the `quadratic_function` objective via `infrastructure.scientific.stability.check_numerical_stability()`.

Dimensional Scaling I

Performance benchmarking spans problem dimensions $d \in \{1, 2, 5, 10, 20, 50\}$, from the scalar case ($d = 1$) to moderate dimensionality ($d = 50$), using identity-Hessian quadratics to isolate algorithmic scaling from problem conditioning effects. The tracked `output/reports/performance_benchmark.json` records fixed inputs, exact objective values, and validation checks. Wall-clock timing is retained only as a runtime diagnostic because host load and hardware make microsecond values unsuitable as pinned reproducibility claims.

Computational Environment I

- **Python**: 3.14.10
- **NumPy**: 2.5.4
- **Platform**: Linux x86_64
- **Generated**: 2026-11-08T05:36:36Z

Pipeline ordering I

Typical `template_code_project` analysis order (see `scripts/pipeline/stage_02_analysis.py` discovery) is:

Pipeline ordering I

1. `optimization_analysis.py` — writes `output/data/optimization_results.csv`, `output/figures/*.png`, and JSON reports under `output/reports/`.
2. `z_generate_manuscript_variables.py` — reads the CSV and YAML, emits `output/data/manuscript_variables.json`, and writes substituted copies to `output/manuscript/` for rendering.
3. `generate_api_docs.py` — refreshes API markdown consumed by documentation targets.

Pipeline ordering I

PDF compilation then reads from `output/manuscript/` so that figure paths and numeric tables match the analysis that just completed.

Relation to figures I

Figure (sec. 4)	Primary inputs
Convergence trajectories	`experiment.step_sizes`, `initial_point`, `tolerance`, `max_iterations`
Step-size sensitivity	Dense α sweep internal to `generate_step_size_sensitivity_plot()`
Convergence rate	Same trajectories as above; tolerance line uses `convergence_tolerance`
Complexity quad panel	One bar chart per row of `optimization_results.csv`
Stability heatmap	`stability_starting_points` $\times$ `stability_step_sizes`
Dimensional benchmark	`benchmark_dimensions`, fixed $\alpha = 0.1$, internal tol 10^{-10}

Relation to figures I

This table is descriptive documentation only; it is not executed as code during the build.